

TGRop: Top Gun of Return-Oriented Programming Automation

Nanyu Zhong^{1,2,3,4}, Yueqi Chen⁵, Yanyan Zou^{1,2,3,4}, Xinyu Xing⁶, Jinwei Dong^{1,2,3,4}, Bingcheng Xian^{1,2,3,4}, Jiaxu Zhao^{1,2,3,4}, Menghao Li^{1,2,3,4}, Binghong Liu^{1,2,3,4}, and Wei Huo^{1,2,3,4}

¹ Institute of Information Engineering, Chinese Academy of Sciences

² School of Cyber Security, University of Chinese Academy of Sciences

³ Key Laboratory of Network Assessment Technology, Chinese Academy of Sciences

⁴ Beijing Key Laboratory of Network Security and Protection Technology

⁵ University of Colorado Boulder

⁶ Northwestern University

Abstract. Given the prevalence of Return-Oriented Programming (ROP) in exploitation, automating ROP has become a cornerstone of security research and education. Many security measures are evaluated against and thus restricted by the practical capability of ROP. However, the ROP automation state-of-the-art approaches have fundamental limitations in their gadget utilization and fall short of delivering the promise.

To overcome these fundamental limitations, we design and implement *TGRop* which advances ROP automation to a new level. *TGRop* can leverage gadgets that operate memory and perform complex arithmetic calculations. By breaking down the entire computation into sub-goals, *TGRop* effectively reduces search space and thus maximizes the utility of the SAT/SMT solver. More importantly, *TGRop* employs a systematic approach to resolving data dependencies and eliminating side effects.

Our thorough measurement shows that *TGRop* outperforms all existing approaches by more than 1.62 - 3.11 times. Additionally, we validate the rationale behind its design via analytical experiments. When running *TGRop* against the newest ROP mitigations, we discovered their weaknesses and reported to vendors.

1 Introduction

Since presented in 2007 [42], Return-Oriented Programming (ROP), has evolved a lot [6, 24] and been prevalent. Over the past several years, it has become a routine in vulnerability exploitation across layers of computer systems, from applications [12] to OS kernels [16], to System-on-Chip [31, 32], and more. Moreover, ROP techniques are no longer limited to stack-based corruption but have also been applied to exploit heap-based corruption [25, 47].

As ROP is powerful, researching gadget chain construction becomes highly valuable. To evaluate the security improvement of defenses such as system call filtering and streamlining [11], critical gadgets removal [13, 29], Address Space Layout Randomization (ASLR), and control-flow integrity (CFI) [1, 7], to which

extent ROP is prevented is widely accepted as a metric. For ASLR, attackers often exploit information leaks or use gadgets from the small non-randomized sections to continue their attack. Recently gaining popularity, CFI restricts the legal range of control flow jumps in each Indirect Control Transfer within a program. This greatly limits the capability of ROP attacks, yet many works [9, 19, 20, 41, 43, 48] have also demonstrated that ROP attacks are still feasible. Others just reduce the number of gadgets in a program, making it harder for manual searching and chaining, but an automated ROP tool should be more suitable to overcome this. Over the past decade, a series of efforts (*e.g.* [4, 10, 23, 34, 38, 41]) have been dedicated to automating gadget chain construction. These tools have evolved to be the cornerstone of security research and education.

However, in this work, we discovered that these state-of-the-art automation tools have fundamental limitations and often fail to deliver the expected outcomes, misleading security analysis. The technical shortcomings of State of the Art (SOTA) approaches are three-fold. First, they fail to leverage gadgets involving memory operations and complex arithmetic for gadget chaining, thus missing valuable gadgets. Second, some methods heavily rely on symbolic execution and SAT/SMT solvers, which can easily result in timeouts in large programs with numerous candidate gadgets. Third, during gadget chaining, SOTA tools employ ad-hoc approaches rather than systematic design to resolve data dependencies and eliminate side effects between gadgets, which results in failure.

To this end, we design and implement *TGRop* to resolve these critical shortcomings, thereby advancing ROP automation to an unprecedented level. The input of *TGRop* is the program’s state at control flow hijack and a specified malicious computation to meet the exploitation target. Based on the specification, *TGRop* generates a blueprint which finally leads to a working gadget chain. Prior to gadget chaining, *TGRop* builds a Gadget Computation Graph (GCG) to include gadgets that perform memory operations and complex arithmetic, which strengthens ROP’s computation capability.

Technically, the blueprint comprises several stages, and within each stage, *TGRop* constructs the gadget chain in four phases. In Phase I, *TGRop* traverses GCG and tentatively chains gadgets to identify data dependencies that require resolution. Phase II focuses on dependency resolution and in Phase III, *TGRop* eliminates side effects between each gadget through topological sorting. Lastly, Phase IV chains gadgets through topological sorting to produce new program states which are used in the next stage of the blueprint. *TGRop* works in a stage-by-stage workflow until finally a complete chain is successfully constructed. By breaking down the entire computation into stages and further into sub-goals, *TGRop* reduces the search space and maximizes the utility of SAT/SMT solvers.

In evaluation, we compared *TGRop* with six SOTA tools, using two datasets across architectures. This large-scale experiment shows that *TGRop* remarkably outperforms all SOTA tools. Further, we conducted analytical experiments to better understand the reason *TGRop* has such an exceptional performance, which validates the rationale of *TGRop*’s design. More importantly, we ran *TGRop* against the newest ROP mitigations that were introduced in OpenBSD

in 2018 and by the GCC and Clang compilers in 2021. We discovered that *TGRop* can bypass both designs for most randomly selected cases. We have reported our findings to vendors and received positive feedback.

In summary, ROP automation is a cornerstone of security research and education. The effectiveness of many security measures is evaluated and correspondingly restricted by the practical capabilities of ROP, though, in theory, ROP is Turing-complete. *TGRop* stands out as the premier and most systematic tool to automate gadget chain construction. It makes the following contributions:

- We developed a systematic approach to automating gadget chain construction, overcoming the fundamental limitations of existing works.
- We implemented the innovative approach as an end-to-end tool named *TGRop* and conducted a thorough evaluation of it. Additionally, we conducted analytical experiments to validate the design rationale. *TGRop* will be open-sourced once the paper is accepted.
- We ran *TGRop* against the newest ROP mitigations. The discovered weaknesses have been reported to vendors.

We make the raw data and detailed information of *TGRop* available: <https://sites.google.com/view/tgropsdetails>.

2 Shortcomings of State-of-the-Art Approaches

Current techniques for automating gadget chain construction fall into three categories: hardcode-based, heuristic-based, and exploratory approaches. Since hardcode-based approaches (*e.g.*, *Ropper* [36] and *ROPgadget* [33]) are straightforward and their shortcomings well-studied [38, 46], in the following, we will put more effort into heuristic-based approaches and exploratory approaches.

Heuristic-based Approaches. Landmarks of this type include *Q* [40], *Angrop* [4], *Ropium* [34], *Exrop* [23], and *RiscyROP* [10]. These tools construct chains typically in three steps.

First, heuristic-based approaches leverage method like directed dependency graphs [34], weakest preconditions computation [42], and symbolic execution [10] to summarize the semantics of gadgets and categorize them accordingly. Next, categorized gadgets are selected heuristically to form chains. They attempt to search and chain the gadgets by leveraging core techniques like MOV-connection graphs [18], depth-first search algorithms [35], or breadth-first search algorithms [4]. Furthermore, various tools apply predefined rules to address dependencies and side effects of selected complex gadgets in the previous step, sometimes discarding them even if they are usable. *Angrop*, *RiscyROP*, and *Exrop* only consider gadgets with basic arithmetic such as `add/sub` and `jmp/call` operations. *Ropium* further includes some straight memory writes and operations over immediate values. Besides, when encountering gadgets with unsupported complex semantics or deemed to have destructive side effects, they are directly discarded (*e.g.*, *Q*, *RiscyROP*).

However, these ad-hoc approaches (discarding gadgets) overlook handling data dependencies or side effects between gadgets to construct more available

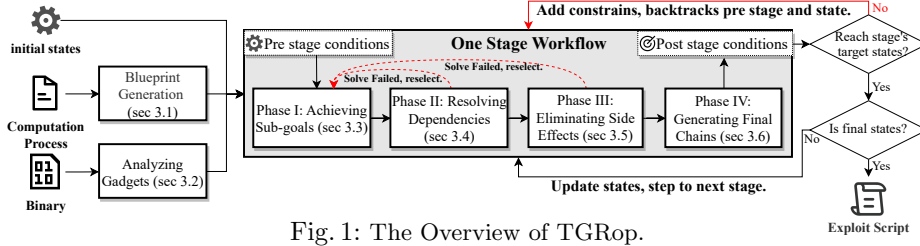


Fig. 1: The Overview of TGRop.

gadget chains. This is especially true for gadgets involving memory-related operations or other complex arithmetic that they cannot use. Though *ROPmate* [2] offers a visualization tool to help the identification of gadget dependencies and side effects, how to automate repairing remains an open problem.

Exploratory Approaches. Unlike heuristic-based approaches which rely on gadget classification, exploratory approaches like *Limbo* [41] and *SGC* [38] heavily use concolic execution and SAT/SMT solver to chain gadget.

Limbo generates gadget chains for programs with CFI protection. It allows users to specify the initial state of registers and memory cells, and then concolically executes [8, 15] gadgets that comply with the Control-Flow Integrity (CFI) policy [1]. By minimizing the distance between the target state and the state after executing a gadget, *Limbo* gradually constructs a gadget chain that meets the exploitation goal. *SGC* converts the semantics and side effects of gadgets into constraints. After fixing the chain length, it randomly selects gadgets into the chain. By querying SAT/SMT solver [44], *SGC* determines if the chain can transition the initial state to the target exploitable state.

Exploratory approaches follow a hypothesis-testing process. During exploration, they will keep trying until timeout or fortunately finding a working solution. Although setting thresholds can reduce overhead, changes in the threshold values can lead to exponential increases in computational effort. Therefore, exploratory approaches are better suited for small programs with limited gadgets.

Challenge. Based on the situations barely well-handled in prior works, we have identified three challenges:

- C1.** How to involve the use of gadgets with memory operations and other complex arithmetic to largely strengthen the ability of ROP computation?
- C2.** How to reduce search space and fully leverage SMT solvers for chaining?
- C3.** How to handle data dependencies and side effects in a systematic approach?

TGRop will show how to address these challenges in the following sections.

3 Methodology

We designed and implemented *TGRop* guided by dynamic programming principles, whose architecture is illustrated in Figure 1. *TGRop* requires that users input the registers and memory’s initial state at control-flow hijack, along with the computation process, for defining the exploit. Firstly, *TGRop* tries to break down the computation process into sub-goals and stages outlined in a blueprint. Thus, the task is redefined as solving a series of gadget chaining challenges for

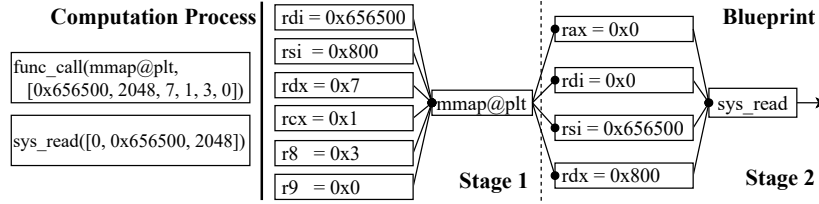


Fig. 2: Computation process of the user input based on user-friendly interfaces, and the blueprint generated by *TGRop* accordingly. This part shows `mmap` memory allocation and reading shellcode with the `read` system call.

each sub-goal. Then *TGRop* sequentially resolves data dependencies and side effects for each gadget, ultimately selecting the optimal gadget chaining strategy for each stage from several valid topological sorting schemes, with minimal alteration to the program state.

Guided by the blueprint, *TGRop* constructs gadget chains in a stage-by-stage workflow, as shown in Figure 1. Starting from the initial state, it forms chains for each stage through a four-phase process, updating states upon reaching targets. If a stage fails due to unsuitable state values, *TGRop* adds constraints, backtracks to adjust previous stages, and regenerates states for subsequent stages. This workflow repeats until a full chain is successfully built for all stages. By dividing the entire computation into stages and solving one stage locally, *TGRop* reduces search space (**C2**). Crucially, this method enables *TGRop* to find solutions where later stage payloads rely on earlier generated states.

Before chaining gadgets, *TGRop* builds a Gadget Computation Graph (GCG) with all potential gadgets, including those involving memory operations and complex semantics to enhance ROP attack capabilities (**C1**). For each stage, *TGRop* first traverses GCG and tentatively chains gadgets and identifies data dependencies with the help of SAT/SMT solver for each sub-goal independently in phase I. Sub-goal division reduces the search space (**C2**) and offers immense flexibility in chaining gadgets, as there is no immediate need to consider the complex side effect relationships between sub-goals. During this phase, each sub-goal has numerous temporary gadget chaining schemes. Phase II iteratively resolves dependencies for each chained gadget by creating new sub-goals, with the aid of an SMT solver (**C3**). Phase III eliminates side effects across gadgets on a global scale (**C3**). If resolving a gadget chain in Phases II or III fails, it disrupts only one of the sub-goal’s chain schemes, necessitating the selection of another one. This process is essentially a form of traversal but is notably efficient. Ultimately, Phase IV involves selecting the most effective topological sorting scheme to chain gadgets and update the program state for the next stage.

3.1 Preprocessing: Blueprint Generation

TGRop automates the generation of a gadget chain that starts from an initial state of registers and memory at control-flow hijack, which is described in the JSON format (e.g., `rax:0x10`). And *TGRop* also provides interfaces to easily define the computation process, covering five kinds of operations: (1) register write, (2) memory read, (3) memory write, (4) function call, and (5) system call.

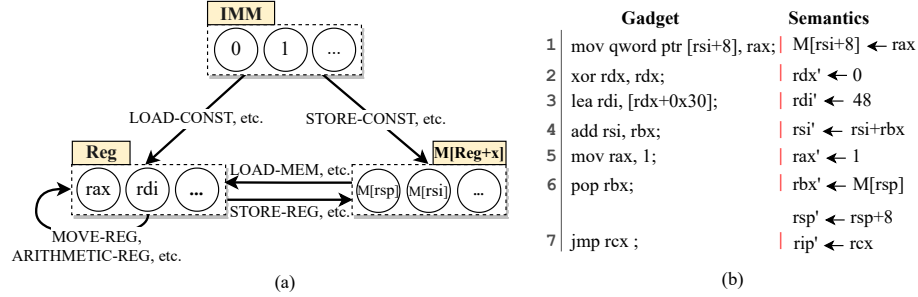


Fig. 3: Figure(a) is the Gadget Computation Graph (GCG) used as a clue for searching gadgets to achieve sub-goal. And figure(b) is the example of gadget semantic extraction.

Using these user-friendly interfaces, *TGRop* users can describe the malicious computation process in a manner similar to drafting a C program.

TGRop parses the computation process and outputs a blueprint for gadget chain generation, as shown in Figure 2. In this blueprint, each node represents a sub-goal that needs to be met by a gadget sub-chain, and the edges depict the order of these sub-goals. A sub-goal could be a calling to the (1), (2), and (3) interfaces in the specification. For (4) function call and (5) system call, *TGRop* goes a step further by creating sub-goals that aim to set arguments before function and system calls. Thus, the blueprint is further divided into stages with the nodes of function calls and system calls as the boundary. Figure 2 demonstrates the two-stage process of specifying the exploitation routine for storing malicious shellcode in memory. Each stage consists of several sub-goals, which must be achieved before invoking a function call or system call.

3.2 Preprocessing: Analyzing Gadgets

In this phase, *TGRop* analyzes all gadgets in the target program to extract and categorize their semantics, including ROP, Jump-Oriented Programming (JOP), and Call-Oriented Programming (COP) gadgets so that *TGRop* can construct these code-reuse attacks. Then, *TGRop* builds a Gadget Computation Graph (GCG) to depict the potential data flow these gadgets can perform, which serves as the building block of the following phases.

Semantic Extraction and Categorization At the very beginning, *TGRop* identifies all gadgets in the target program using *Ropper* and *ROPgadget*. Given a gadget, *TGRop* symbolizes all registers and memory cells and performs symbolic execution from the first instruction. It accurately captures all modifications the gadget can make to the program state and represents them as symbolic expressions. Notably, *TGRop* includes conditional branch instructions in gadget chaining by forking execution when encountering a symbolized conditional branch. Figure 3(b) showcases the semantic extraction.

We categorize all instruction semantics into 15 groups, as listed in Table 3 in the appendix. The categorization is based on the type of destination operands, *e.g.*, general-purposed registers, memory cell, and PC register, with further sub-classification according to source operand types, including immediate values,

system call numbers, parameters for `call` instructions, branch condition, and conditional jump target for `cmp-jne`-like instruction twins. The semantics categorized here are much more complete compared with prior works (*e.g.*, *Q*). As we will discuss soon, this richness helps avoid omitting valuable gadgets when constructing gadget chains, especially when a specific gadget is a unique solution.

Gadget Computation Graph Construction The Gadget Computation Graph (GCG) is designed to systematically organize all gadgets in the program by their potential data flow contributions. *TGRop* leverages GCG as the cue to search for the single gadget or combinations that can set registers and memory cells. These registers and memory cells could serve as arguments for function calls and system calls or be used to resolve dependencies and eliminate side effects.

As illustrated in Figure 3(a), the nodes in the GCG represent operands that could appear in an x86 instruction set. For simplicity, we grouped them into three types: (1) immediate values, (2) general-purpose registers, and (3) memory cells. The directed edges between nodes represent the data flow direction of gadget’s semantic. Note that we don’t include PC into the GCG as its role in directing execution, can be achieved by setting register or memory cells, thus making its inclusion in the GCG unnecessary. We will be further explained in Phase I.

In the construction process of the GCG, the edges represent a deeper classification of gadgets based on their semantics at the operand level. We analyze each gadget’s semantics, focusing on its source and destination operands, to assign them to corresponding edges in the GCG. In cases with multiple source operands, these are individually classified, creating several edges from each source operand to the destination operand. To this end, all gadgets in the target program can be associated with one or more edges in the graph. For example, in Figure 3(b), the execution of `mov qword ptr [rsi+8], rax` and `xor rdx, rdx` implies the presence of LOAD-CONST and STORE-REG semantics. This results in associations with the `rax -> M(rsi)` and `0 -> rdx` edges in the GCG.

3.3 Phase I: Achieving Sub-goals

Starting from each stage, *TGRop* selects or combines gadgets to meet sub-goals in the blueprint independently. At this moment, dependencies and side effects between gadgets for different sub-goals are not considered, being addressed later in Phases II and III. Since invoking function calls and system calls can be naively achieved via gadgets ending with `ret`, `jmp`, `call` and `syscall` which get the target address from the controllable stack, memory cells or register, we will put more effort into describing how *TGRop* leverages GCG to set registers and memory cells as desired values. These are then employed as arguments for system and function calls and to resolve dependencies, which will be elaborated in Phase II.

Start from GCG Traverse Typically, a sub-goal cannot be directly achieved through one single gadget. Therefore, gadgets need to be combined to fill in semantic gaps. With the GCG, searching for suitable gadget combinations is converted to a reachability problem between graph nodes. More specifically, given a sub-goal, for example, setting `r8` register as `0x3`, *TGRop* traverse the GCG to identify all nodes and corresponding paths that can reach `r8` register. Since

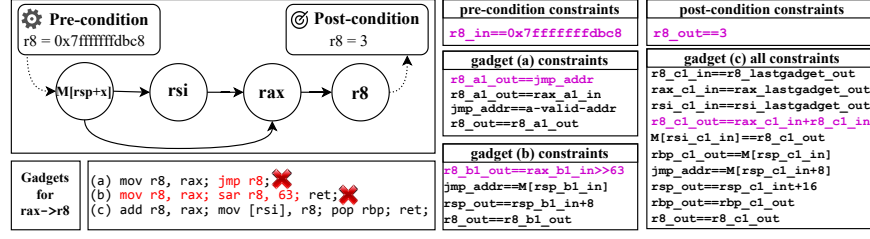


Fig. 4: Example of GCG traverse and tentative chaining in Phase II for the sub-goal of setting `r8` as 3. For the `rax->r8` edge, gadgets (a) and (b) are discarded immediately while (c) is up in the air before dependencies are resolved in Phase II.

the edges in GCG represent potential data flow that gadgets can perform, the nodes and paths that cannot reach `r8` node are definitely unable to modify `r8`. Therefore, ruling them out will not have negative impacts.

TGRop performs this traverse using a depth-first search framework and outputs a set of paths. Every edge in these paths is associated with at least one gadget. Example paths of `r8=8` sub-goal as shown in Figure 4, could be `M[rsp+x]->rsi->rax->r8` or `M[rsp+x]->rax->r8`, which will be discussed soon.

Tentative Chaining *TGRop* employs an SMT solver to determine which gadgets in the GCG path might aid in achieving sub-goals. It first incorporates the initial state (*i.e.*, pre-condition) and the sub-goal (*i.e.*, post-condition) as constraints into the solver. It then goes through all paths, converting the semantics of gadgets associated with the edges into constraint expressions. These constraints will be conjuncted with pre-condition and post-condition and independently fed into the solver. If the constraints for a gadget chain are not feasible, *TGRop* discard this chain and tries the next one.

For example, in the path `M[rsp+x]->rsi->rax->r8`, GCG provides three gadgets for the edge of `rax->r8`, with z3 constraints shown in Figure 4. Note that the constraints of individual gadget are linked by `&`, and those between gadgets on the same edge are connected by `|`. For the first two gadgets, the SMT solver identifies and eliminates due to conflicts with post-condition constraints: the first one has a constraint `r8_out==jmp_addr` which conflicts with the sub-goal `r8_out==3`, as it should be an address of the next gadget; and the second's gadget, it contains a z3 constraint `r8_out==rax_b1_in>>63` limits the `r8` register to 0 or 1 (right-shifting a 64-bit value by 63 bits equals division by 2^{63}), conflicting with the sub-goal `r8_out==3` and making it unfeasible. For the last gadget, the `rax_c1_in` register, stems from the controllable stack, can be any value. The constraints of `r8_out==rax_c1_in+r8_c1_in` means we could control `r8_out`'s value depending on `r8_c1_in` from either the initial state or the execution of prior gadgets. Tentative chaining cannot conclusively discard it at this moment. Instead, *TGRop* will step to Phase II to resolve dependencies (§3.4).

To reduce the burden of the solver, we introduce optimizations in the above procedure. More specifically, instead of chaining gadgets from the start to the end of the path, *TGRop* performs in reverse order. In other words, the gadgets that are close to the sub-goal in the path are chained first. Intuitively, the gadgets

near the start of the path are developing the state and can seldom encounter infeasibility problems. In comparison, the reverse order enables *TGRop* to quickly discard gadgets that certainly cannot satisfy the sub-goal (*e.g.*, setting `r8` as 3). This pruning not only reduces the total number of gadgets for tentative chaining but also allows the solver to terminate earlier.

3.4 Phase II: Resolving Dependencies

Dependencies of a gadget refer to when a register or memory cell needs a specific value or value range for the gadget to meet a sub-goal. To resolve dependencies, we typically create new sub-goals, such as setting `r8_c1_in` to 3 and `rax_c1_in` to 0, and include them in the blueprint. These new sub-goals should be executed before the original ones to ensure the correct value setting. However, while chaining gadgets to achieve these new sub-goals, additional dependencies might arise, prompting a repeat of the sub-goal creation process. This resolution method is iterative and continues until all dependencies are addressed or the process fails due to the lack of suitable gadgets. To prevent infinite loops in dependencies, a worklist is maintained. For instance, if sub-goal A leads to sub-goal B, creating sub-goal C identical to A, *TGRop* won't add C to the worklist, avoiding a cycle. However, sub-goal C becomes unattainable in this gadget chain.

Three types of sub-goals will be created for dependency resolution:

- **Valid Address:** This sub-goal ensures that the value stored in a register, after an arithmetic operation, *e.g.*, add or subtract, is a valid address. As such, the operation `M[reg]` which either reads from or writes to the address indicated by `reg` can carry on without crashing the program.
- **Correct Target:** This sub-goal ensures that the next instruction to execute is the start of desired gadgets. To resolve this, the operand must be set to the address of the correct target. In addition, for IF semantics, the condition predict should be evaluated to `true` if the desired next gadget lies within the true branch, or else to `false`.
- **Suitable Value:** This sub-goal ensures that the value of a source operand is suitable so that after computation, the value of a destination operand is correct. Still taking the third gadget of the `rax->r8` edge in Figure 4 as an example, given that the `rax` register steams from the controllable stack and can be any value (*e.g.*, 3), *TGRop* will create a sub-goal of setting `r8` as an immediate value like 0 so that `rax+r8` can be feasibly computed as 3.

After creating a new sub-goal, it's crucial to determine its effective values. In most cases, using a solver to determine the values for each sub-goal is the fastest approach. However, when the solver's suggested values can't be matched with available gadgets in the program, *TGRop* will first consider a certain value from GCG's LOAD-CONST type paths (from immediate to the operand). For example, if the solver suggests setting `r8` as an uncommon value like 0xeeeff, it's unlikely we will find a gadget that can meet this sub-goal, so *TGRop* looks for a gadget containing `xor rd8, r8d`; to zero `r8` as GCG includes the path of `0->r8`.

3.5 Phase III: Eliminating Side-Effects

After Phase I and II, *TGRop* obtains a set of gadget chains for each sub-goal in the stage. The key challenge in this phase is determining the execution order of

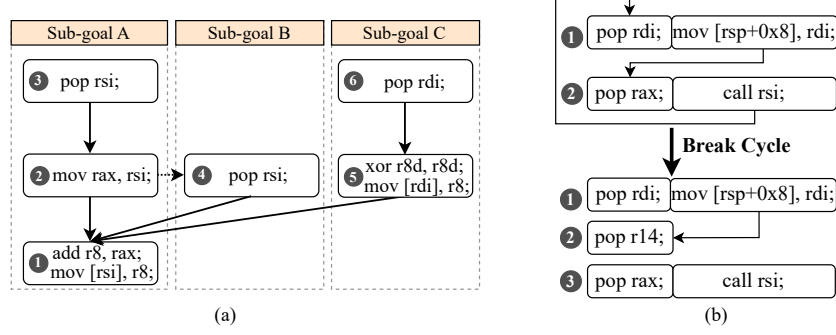


Fig. 5: Fig.(a) is the example of topology built in Phase III to eliminate side effects. It involves three sub-goals: Sub-goal A is originally present in the blueprint, and Sub-goals B and C are created for dependency resolution in Phase II. The direction of the edge indicates the desired execution order or have write-read pair should be protected. Fig.(b) is an example of manipulating stack to break cycles. For simplicity, all **ret** instructions at the end of gadgets are omitted.

gadgets between sub-goals to eliminate potential side effects. The root cause of these side effects stems from earlier chain executions updating the initially static state, leading to certain chains failing to work properly. Analytically, to avoid side effects, we need to ensure that write-read pairs between and within the sub-goal chain are never broken during assembly. More specifically, a write-read pair refers to a situation where an earlier gadget writes to a register or memory cell that a later gadget reads. This order cannot be reversed. Otherwise, the value obtained by the later gadget becomes incorrect. Similarly, a third gadget that sets the same register or memory cell cannot be inserted in between either.

TGRop's solution is essentially sorting a topology derived from the blueprint. For example, Figure 5(a) contains three sub-goals. The sub-goal A is to set **r8** as 3 - a sub-goal we've been discussing so far. The sub-goal B is to set **rsi** as a valid address so that gadget 1 in the first sub-goal will not crash the program. The sub-goal C is to set **r8** as 0. Both sub-goals B and C are created in Phase II for resolving dependencies. Within each sub-goal, gadgets are executed in their original orders. Between sub-goals, there are two write-read pairs concerning **rsi** and **r8**'s value, from gadgets 4 and 5 to gadget 1, independently. Thus, gadgets 4 and 5 must be executed before gadget 1. The last execution order to determine involves gadgets 2, 3, and 4 since they either write or read from **rsi**. On the one hand, gadget 4 cannot be executed before gadget 3. Otherwise, the write-read pair between gadget 4 and gadget 1 is broken. On the other hand, gadget 4 cannot be executed between gadget 3 and 2 because it will break the write-read pair between them. Therefore, gadget 4 must be executed after gadget 2. With the topology constructed based on the above order, in Phase IV, *TGRop* can perform topological sorting to assemble gadgets without concern for side effects.

For a single stage, *TGRop* can build a series of such topologies. This is because, even for one sub-goal, Phases I and II will construct many gadget chains that can work independently, and any of these chains can be chosen by *TGRop* for assembly. For topologies devoid of cycles, *TGRop* can proceed directly to

Phase IV. However, for those containing cycles, additional manipulations are needed. For example, as shown in Figure 5(b), a cycle formed by two gadgets involves stack-based write-read pairs. To break such cycles, *TGRop* employs stack adjustments, such as inserting a gadget for an extra `pop` instruction to change the stack pointer (`rsp`), thereby disrupting the cycle. This technique is also applicable for breaking longer cycles.

3.6 Phase IV: Generating Final Chains

With the topologies generated in Phase III, *TGRop* will perform topological sorting to generate gadget chains for the stage. After each chain transitions from an initial state into a new state, *TGRop* steps into the next stage, repeating Phases I, II, and III to produce another chain. Finally, *TGRop* assembles chains for all stages to complete the entire blueprint.

During a single stage’s topological sorting, *TGRop* optimizes the chain by pruning redundant gadgets. In previous phases, gadgets are chained to meet individual sub-goals, focusing solely on eliminating side effects. This may include gadgets that unintentionally fulfill multiple sub-goals, such as a gadget chosen for setting `r8` to 3 (sub-goal D) and also setting `rdx` to 7 (sub-goal E), thus rendering the gadget for the sub-goal E redundant. *TGRop* carefully evaluates if the next gadget will modify the current state and makes pruning decisions to maintain efficiency. For all chains produced in a single stage, *TGRop* prioritizes using shorter gadgets, especially in terms of instruction count and stack usage, to form the initial state for the subsequent stage. This approach helps to circumvent the limitations in stack control range posed by certain vulnerabilities.

4 Implementation

TGRop is implemented using Python, comprising a total of 12,886 lines of code, not including comments. *TGRop* will be open-sourced under MIT License once accepted. We have created an anonymous Github repo [5] for submission.

TGRop is built on top of existing libraries and tools. In Phase I, all gadgets in the target program are identified using *Ropper* [36] and *ROPgadget* [33]. The symbolic execution engine used to extract semantics is *angr* [3], which can analyze gadgets across different architectures. Additionally, *TGRop* supports leveraging the GDB [14] Plugin to automatically collect register and memory values as the initial state for gadget chain construction. In Phases II, III, IV, and V, *TGRop* uses the Z3 solver [49] to discard unsuitable gadgets, resolve dependencies, avoid side effects, and optimize gadget chains.

5 Evaluation

Now, we thoroughly evaluate *TGRop* against six open-source gadget chain automation tools. Through analyzing experiment results, we aim to answer three research questions: To what extent does *TGRop* outperform SOTA approaches? (**RQ1** in §5.2); Why does *TGRop* outperform SOTA approaches? (**RQ2** in §5.3); Has *TGRop* discovered weaknesses in newest mitigations? (**RQ3** in §5.4).

5.1 Experiment Setup

Baselines. We included six open-source SOTA tools with the higher star counts on GitHub as baselines: *Ropper* [36] and *ROPgadget* [33] in the category of hardcoded-based approach, *Angrop* [4] (the most popular automated tool on GitHub), *Exrop* [23] and *Ropium* [34] in the category of heuristic-based approach, and *SGC* [37] in the category of exploratory approach. We didn’t include *Limbo* [41] because it is not open-source. For most experiments, *SGC* is used in the default configuration - alternative configurations will produce even worse results as we will show in §5.3. All baselines, along with our own *TGRop* are modified to be compatible with ROP Benchmark [45] framework. This work includes multiple program test suites and can automatically trigger the gadget chain construction tool to produce chains for the stack buffer overflow vulnerabilities injected into the programs of these benchmarks, thereby disabling ASLR.

ROP Goals. In the experiment, we consider six ROP goals across three kinds: (1) get a shell by invoking `execve("/bin/sh", 0, 0)`, (2) write an arbitrary string to an arbitrary memory location, and (3) control three to six (maximum number required for system calls) registers. The three goals represent the most desired capability by attackers - code execution and arbitrary write. To automatically determine if the goal is met during our large-scale experiment, we enhanced the validation module in the ROP Benchmark using a QEMU-based approach, enabling support for more architectural targets. The only exception to this approach is the *SGC* baseline, which has its own solver-based validation module.

Machine and Timeout. All experiments were run on an Ubuntu 20.04 machine with an Intel i9-10900X 3.70GHz 20-core and 128GB RAM. Baselines and *TGRop* are given one hour per program to generate gadget chains for each goal, except in RQ3 to compare *TGRop*’s advantage over exploratory tools.

5.2 Performance of *TGRop*

We evaluated the overall performance of *TGRop* using two datasets. One is all x64 programs shipped in Debian 10, CentOS 7, and OpenBSD 6.2, from the ROP Benchmark⁷. Another includes firmware and libraries from different architectures: MIPS, ARM, and PowerPC. We utilized this dataset to test the generality of *TGRop*’s design.

OS Distros from ROP Benchmark Table 1 highlights *TGRop*’s significant success rate over SOTA baselines across three chain goals (other three goals results in Table 4 in Appendix), with *ROPgadget* and *Ropper* unable to meet goals #2 and #3. Notably, for goal #1, *TGRop* achieves a 97.69% success rate, significantly outperforming baselines with rates of 3.57% to 60.23%, making it 1.62 to 27 times more effective. *TGRop* only has two timeouts for `libcudata.so.63.1.bin` in Debian 10 and `clang.bin` in OpenBSD 6.2. This is because both programs are large libraries exceeding 26MB, containing over 11,6104 candidate gadgets. As such, symbolic execution in gadgets analysis fails to extract the

⁷ We remove programs that don’t make system calls since one of our ROP goals is to invoke `execve` to get a shell.

Table 1: Results of *TGRop* and SOTA tools over ROP Benchmark. The “X/Y/Z” format indicates that for X programs, the tool successfully generates chains meeting the goal; for Y programs, chains fail validation; for Z programs, the tool fails due to timeout; and for the remaining programs, generation fails within 1 hour. NA implies the tool does not distinguish between success and failure in its output.

	Debian 10	CentOS 7	OpenBSD 6.2	TOTAL	Debian 10	CentOS 7	OpenBSD 6.2	TOTAL	Debian 10	CentOS 7	OpenBSD 6.2	TOTAL
Program #	139	121	87	347	139	121	87	347	139	121	87	347
	Goal #1: Get a Shell				Goal #2: Arbitrary Write				Goal #3: Set Three Registers			
ROPgadget	5/0/0	4/0/0	4/0/0	3.75%	NA	NA	NA	/	NA	NA	NA	/
Ropper	53/NA/0	31/NA/0	17/NA/1	29.11%	NA	NA	NA	/	NA	NA	NA	/
Exrop	55/3/6	48/6/9	6/7/12	31.41%	67/4/5	72/7/6	7/20/9	42.07%	85/0/1	71/1/2	38/0/6	55.91%
Angrop	98/11/2	70/5/1	33/4/5	57.93%	85/4/29	90/3/5	48/1/12	64.27%	58/3/22	54/5/3	19/3/9	37.75%
ROPium	100/12/1	66/9/0	43/6/1	60.23%	122/5/0	100/1/0	57/3/2	80.40%	104/5/0	73/3/0	43/4/1	63.40%
SGC	17/NA/38	20/NA/30	9/NA/33	13.26%	25/NA/54	18/NA/83	15/NA/45	16.71%	58/NA/41	63/NA/38	60/NA/24	52.16%
TGRop	138/0/1	117/0/0	84/0/1	97.69%	138/0/1	121/0/0	86/0/1	99.42%	138/0/1	121/0/0	86/0/1	99.42%

Table 2: Cross-Architecture results of *TGRop* for goal #1. The fourth column presents the number of programs that at least have one valid GCG path with an exploitation target. The meaning of “X/Y/Z” is the same as that in Table 1.

ARCH	Target	# Files	# Files with GCG-path	# Success	ARCH	Target	# Files	# Files with GCG-path	# Success
MIPS	Archer A54v1	78	45	36/0/0	MIPS	debian mipsel	624	272	223/0/14
MIPS	blink X12	74	30	25/0/0	ARM	NETGEAR R8500	7	7	6/0/0
MIPS	Cisco RV110W	218	110	75/0/4	ARM	Tenda G3	7	7	5/0/0
MIPS	TL-WR841Nv14	59	31	24/0/0	ARM	Tenda W20EV4	6	6	3/0/0
MIPS	TL-WR902ACv3	82	42	33/0/0	PowerPC	libc-2.31.so	1	1	1/0/0

semantics of all candidates within the given 1 hour. For the other two goals, *TGRop* also remarkably outperforms all baselines.

Heuristic-based approaches have shown success rates ranging from 31.41% to 63.40%, which are influenced by the completeness of rules. In contrast, *TGRop* leverages semantic extraction and categorization, along with GCGs, to enable more effective gadget utilization. For exploratory approaches, *SGC* has the most timeouts which lowers its success rate. Despite more time, *SGC* underperforms compared to *TGRop*, which uses less symbolic execution and solver techniques. We will demonstrate *TGRop*’s advantages with empirical evidence in §5.3.

Compared with Human Efforts. In particular, we conducted a human study to justify the necessity of automation. The study has received IRB approval from the institute (§A.1). We recruited nine participants who have at least two years of CTF experience in pwn. We randomly selected 10 programs from our dataset and each participant has one hour per program to complete the goal #1 to get a shell. It turns out that human participants constructed a gadget chain for at most seven programs, whereas *TGRop* succeeded in all 10, averaging only 5 minutes each. This huge gap demonstrates that the capability of *TGRop* significantly outperforms human beings. More detailed experiment results are in Table 5.

Cross-Architecture Firmware and Libraries Unlike existing approaches, *TGRop* is designed to be independent of architecture. To evaluate this aspect, we tested *TGRop* on IoT device firmware and libraries across MIPS, ARM, and PowerPC architectures. The results for goal #1 presented in Table 2, show *TGRop* successfully constructed gadget chains for 431 out of 1,156 programs. The success rate is lower than that observed with x64, mainly because only 551 programs contain the required GCG paths. This limitation arises from the

RISC nature of MIPS, ARM, and PowerPC architectures, which restrict the use of unaligned instructions in gadget chain construction. Consequently, gadget chains tend to be longer in RISC than in CISC architectures like x64, with an average chain length of 724.54 for MIPS compared to 120.99 for x64.

5.3 Ablation Analysis of *TGRop*

To have a deeper understanding of why *TGRop* outperforms heuristic-based tools and exploratory tools, we conducted ablation analysis.

Compared with Heuristic-based Tools We performed two experiments to draw comparison between heuristic-based tools and *TGRop*: (1) analyzing the distributions of gadget types used for chain construction. (2) analyzing cases in OpenBSD 6.x that *TGRop* succeeded but heuristic-based tools failed. The two experiments show that, *TGRop* outperforms because of the design of CGC and efforts on resolving dependencies.

CGC Helps. In this experiment, we treated `rdi`, `rsi`, and `rdx` as destination operands, and gathered relevant gadgets from the GCG to evaluate *TGRop*. For comparison, we applied the heuristics of *Angrop*, *Ropium*, and *Exrop* individually to the target program and categorized all gadgets by source operand types - stack, registers, immediate value, and non-stack memory. The results, detailed in Table 6 in the Appendix, reveal that heuristic-based tools utilize only 18.34% of gadgets per program, predominantly for controlling the PC and setting destination registers or memory cells without using arithmetic semantics. *TGRop*, however, can almost fully leverage all program gadgets, owing to the comprehensive construction of the GCG. This extensive use of gadgets, including an additional 81.66% compared to heuristic tools, empowers *TGRop* to efficiently resolve critical dependencies, particularly for memory operations, greatly enhancing its chain construction capability.

Resolving Dependencies Matters. To elucidate the limitations of heuristic-based tools, we manually evaluated 202 programs’ payload (from OpenBSD 6.2, OpenBSD 6.4 and OpenBSD 6.5) where only *TGRop* was successful for goal #1. This examination revealed that *TGRop*’s proficiency in resolving dependencies (challenge **C3**) is pivotal. From these programs, 19 were successful due to resolving “Valid Address” issues, 89 due to “Correct Target” and 183 due to “Suitable Value” concerns. Specifically, *TGRop* adeptly uses memory operations and complex arithmetic, not just `add` or `sub`, to set appropriate register values, respectively succeeding in an additional 75 and 35 programs. Moreover, *TGRop* can leverage ARITHMETIC-STORE-REG semantics, combining memory and arithmetic operations, a feature present in 170 programs and unattainable by heuristic-based tools. These findings demonstrate *TGRop*’s advanced capability in utilizing a wider range of gadget semantics to resolve intricate dependencies.

Compared with Exploratory Tools To explore *TGRop*’s advantages over *SGC*, we conducted two experiments: one extending *SGC*’s time and another increasing gadgets variety. The experiments revealed that neither additional time

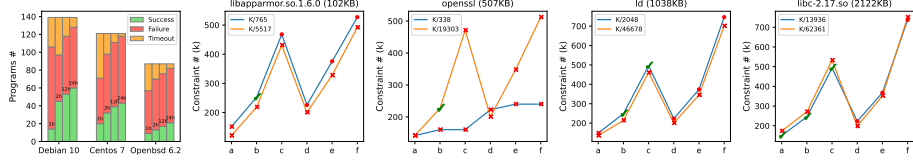


Fig. 6: Subfigure 1: *SGC* with more time. Others: *SGC* with extra gadgets. In four line charts, the y-axis is the number of constraint expressions (in thousand k) for *SGC* on four programs, the x-axis shows six threshold settings: a) $C=4$, $K=300$; b) $C=4$, $K=500$; c) $C=4$, $K=1000$; d) $C=6$, $K=300$; e) $C=6$, $K=500$; f) $C=6$, $K=1000$, all with a 24-hours limit. “K/X” in the legend means *SGC* randomly selects K gadgets from X candidates. ✓ for success, ✗ represents failure, • means timeout or returned UNKNOWN.

nor a richer gadget pool significantly improves *SGC*’s performance compared to *TGRop*, underscoring *TGRop*’s proficiency in challenge C2.

Before describing the two experiments, we introduce three threshold configurations in *SGC*: (1) T - the timeout limit, (2) C - the maximum length of the gadget chain, and (3) K - the number of gadgets considered for chaining in each chain attempt. Within T hours, *SGC* keeps sampling K gadgets from the candidate pool and searching for a chain of C gadgets that can meet the goal.

Time is Not the Key. One can imagine that, since *SGC* heavily uses symbolic execution and SAT/SMT solver, its performance is influenced by the timeout threshold. To this end, we tuned T and re-ran the experiment over ROP benchmark for goal #1. The results are displayed in the first subfigure of Figure 6. By using the default configuration of $C=4$ and $K=300$, allocating more time (1h, 2h, 12h, and 24h) to *SGC* can increase its success rate. Even given 24 hours, timeouts cannot be avoided and the results do not match the performance of *TGRop* achieved within just one hour. Therefore, we can confidently conclude that *TGRop* outperforms *SGC* not because *SGC* is not given enough time.

Solving In Local Reduces Search Space. Ruling out time, we examined whether the richness of candidate gadgets is a determining factor. *SGC* only extracts gadgets that are instruction aligned, 338 gadgets were extracted in the `openssl` program, but after manual analysis, none of them were useful for meeting goal #1. To this end, we provided *SGC* with the 19,303 gadgets extracted by *TGRop*, set T to 24 hours, and adjusted C and K to explore how *SGC* behaves when given richer gadgets. We selected four programs of different sizes, from 102KB to 2212 KB, as representatives and showed their results in Figure 6.

From the figure, the number of constraint expressions (in the unit of CNF) is not correlated with the richness of candidate gadgets. Notably, the `openssl` program followed a similar pattern to other programs, maintaining constraint expressions in the hundreds of thousands. In fact, the performance of *SGC* is even poorer when provided with richer gadgets because the likelihood of sampling the right gadget from a larger pool is lower. Therefore, we concluded that the richness of gadgets is also not the key reason *TGRop* outperforms *SGC*. Further, the number of constraint expressions is affected by the values of C and K, rising from 140k to 740k as these values increase. Given this scale, *SGC* could easily fail or time out. In comparison, the number of constraint expressions for *TGRop* is kept

around 2k, significantly smaller by two orders of magnitude. *TGRop* achieves this by segmenting the gadget chain process into stages and sub-goals, allowing it to focus on localized sub-goals with manageable constraint expressions, effectively addressing challenge **C2**.

5.4 New Findings and Real-world Impact

We used *TGRop* to discover weaknesses in recent ROP mitigations, including compilation options [26–29] adopted by OpenBSD in 2018, and the “-fzero-called-regs” compiler option introduced by GCC and Clang in 2021 [13, 50].

OpenBSD Mitigations. This mitigation, implemented in OpenBSD since version 6.3, decreases gadgets ending with ‘ret’ by reducing the usage priority of the rbx register (associated with the `\xc3` bytecode) and minimizes the occurrence of redundant code, further reducing gadgets in protected programs. A recent study [26] shows that, compared with version 6.2, version 6.5 has significantly fewer gadgets. To illustrate, within `sshd`, the number of unique gadgets has been reduced from 7,650 to 1,843.

Despite this, when run against in total 240 programs in OpenBSD 6.5, *TGRop* was able to generate gadget chains for 185, 231, and 231 programs to achieve goals #1, #2, and #3 respectively. This indicates the mitigation’s marginal effect. Upon closer examination of cases where *TGRop* proved successful, we found that this mitigation primarily removes gadgets ending with `ret` yet overlooks those ending with `jmp` and `call`, particularly memory-related gadgets.

GCC/Clang Mitigations. This compiler option clears callee-used registers upon function return, significantly decreases the number of gadgets that set these registers, such as `pop rbp; ret`. It has been adopted in the compilation process for Linux kernel [21] and Openssh [30], with a reported reduction of at least 20% [21, 22] in gadgets. Adopting this mitigation in the compilation process, we used GCC version 13.2.0 to compile 50 programs from four categories of popular open-source projects, including server utilities such as OpenSSH and Apache, applications like zip, language interpreters like python, and critical library programs. We observed that this option eliminated 60% of the gadgets.

Unfortunately, from the 50 programs, *TGRop* generated gadget chains for 45, 46, and 47 programs to fulfill goals #1, #2 and #3 respectively, indicating that this newest mitigation technique falls short of producing the anticipated security outcomes. Analyzing the cases where *TGRop* was successful, we discovered that this mitigation doesn’t account for arithmetic and memory operations which *TGRop* can leverage to set arguments for function calls and system calls.

Summary and Community Feedback. The effectiveness of ROP mitigations is evaluated and thus restricted by the ability of ROP in practice, though theoretically, ROP is Turing-complete. Current mitigations reduce individual gadgets in an ad-hoc manner, which underscores the need for more systematic ROP automation tools like *TGRop*. In comparison, SOTA tools failed in most cases (Table 7 and Table 8 in appendix). We’ve reported our findings to OpenBSD, GCC, and Clang communities, from whom we have received positive feedback.

6 Discussion

Though *TGRop* outperforms SOTA approaches, it has certain limitations in stack pivoting and CFI bypass. Stack pivoting, which is an exploitation technique to pivot the stack to another memory location that contains several malicious gadgets to reach the goal, is mainly used for heap-based vulnerabilities (e.g., [47]). *TGRop* can add a stack pivot interface into the specification, and add paths like “`X -> rsp`” (`X` could be any node in GCG) into GCG to search for stack pivot gadgets, representing an engineering effort. For CFI bypass, *TGRop* doesn’t assume CFI because the focus of *TGRop* is automation rather than circumvention. However, *TGRop* could enhance CFI bypass capabilities based on previously proposed CFI bypass techniques, such as Counterfeit Object-oriented Programming [9, 39] and Data-Oriented Programming [17]. Since CFI restricts the target address range of each indirect `call/jmp`, based attack concepts like COOP and DOP, could extract a set of usable gadgets. With this set, *TGRop* can systematically search for gadget combinations using the capabilities of GCG, thereby enhancing exploitation capabilities. We believe *TGRop*’s strength lies in its systematic approach to searching for gadget combinations without discarding any potentially usable gadgets.

7 Conclusion

This work addresses the fundamental limitations of current approaches and thus advances the automation of ROP construction to a new level. Thorough experiments show that *TGRop* outperforms all open-sourced state-of-the-art tools and analytical experiments validate the rationale behind *TGRop*’s design. More importantly, *TGRop* discloses design weaknesses in the newest ROP mitigations, which have been reported to vendors. In the future, we will further strengthen *TGRop* by including support for stack pivot and CFI bypassing.

Acknowledgements The authors would like to thank the anonymous reviewers for their valuable feedback. This work is partly supported by National Key R&D Program of China under Grant #2022YFB3103900, Strategic Priority Research Program of the CAS under Grant #XDCO2030200 and Chinese National Natural Science Foundation (Grants #62032010, #62202462).

References

1. Martín Abadi, Mihai Budiu, Ulfar Erlingsson, and Jay Ligatti. Control-flow Integrity Principles, Implementations, and Applications. *ACM Transactions on Information and System Security (TISSEC)*, 2009.
2. Marco Angelini, Graziano Blasilli, Pietro Borrello, Emilio Coppa, Daniele Cono D’Elia, Serena Ferracci, Simone Lenti, and Giuseppe Santucci. Ropmate: Visually Assisting the Creation of ROP-based Exploits. In *2018 IEEE Symposium on Visualization for Cyber Security (VizSec)*, 2018.
3. Angr. Angr github. <https://github.com/angr/angr>. Last updated on Sep 1, 2023.

4. Angrop. Angrop Github. <https://github.com/angr/angrop>. Last updated on Mar 23, 2024.
5. Anonymous. TGRop Github. <https://anonymous.4open.science/r/TGRop-BE8F>. Last updated on Oct 17, 2023.
6. Erik Bosman and Herbert Bos. Framing Signals-a Return to Portable Shellcode. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014.
7. Nathan Burow, Scott A Carr, Joseph Nash, Per Larsen, Michael Franz, Stefan Brunthaler, and Mathias Payer. Control-flow Integrity: Precision, Security, and Performance. *ACM Computing Surveys (CSUR)*, 2017.
8. Sang Kil Cha, Thanassis Avgerinos, Alexandre Rebert, and David Brumley. Unleashing Mayhem on Binary Code. In *2012 IEEE Symposium on Security and Privacy*, 2012.
9. Kaixiang Chen, Chao Zhang, Tingting Yin, Xingman Chen, and Lei Zhao. {VScape}: Assessing and escaping virtual call protections. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1719–1736, 2021.
10. Tobias Cloosters, David Paaßen, Jianqiang Wang, Oussama Draissi, Patrick Jauernig, Emmanuel Stapf, Lucas Davi, and Ahmad-Reza Sadeghi. RiscyROP: Automated Return-Oriented Programming Attacks on RISC-V and ARM64. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses*, 2022.
11. Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, and Vasileios P Kemerlis. Sysfilter: Automated System Call Filtering for Commodity Software. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020.
12. Stefan Esser. Utilizing Code Reuse/ROP in PHP Application Exploits. *BlackHat USA*, 2010.
13. GCC. GCC 11.1.0 Optimize Options. <https://gcc.gnu.org/onlinedocs/gcc-11.1.0/gcc/Optimize-Options.html#:~:text=-fzero-call-used-regs%3Dchoice>. Last updated on Apr 27 2021.
14. GDB. GDB: The GNU Project Debugger. <https://www.sourceware.org/gdb/>. Last updated on Mar 14 2024.
15. Patrice Godefroid, Michael Y Levin, David A Molnar, et al. Automated Whitebox Fuzz Testing. In *NDSS*, 2008.
16. Amrita Milind Honap and Wonjun Lee. Hiding Kernel Level Rootkits Using Buffer Overflow and Return Oriented Programming. In *Information Systems Security: 13th International Conference, ICISS 2017*, 2017.
17. Hong Hu, Shweta Shinde, Sendroiu Adrian, Zheng Leong Chua, Prateek Saxena, and Zhenkai Liang. Data-oriented programming: On the expressiveness of non-control data attacks. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 969–986. IEEE, 2016.
18. Ralf Hund, Thorsten Holz, and Felix C Freiling. Return-oriented Rootkits: Bypassing Kernel Code Integrity Protection Mechanisms. In *USENIX security symposium*, 2009.
19. Kyriakos K Ispoglou, Bader AlBassam, Trent Jaeger, and Mathias Payer. Block Oriented Programming: Automating Data-only Attacks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.
20. Yuan Li, Mingzhe Wang, Chao Zhang, Xingman Chen, Songtao Yang, and Ying Liu. Finding Cracks in Shields: On the Security of Control Flow Integrity Mechanisms. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.

21. Linux. Linux kernel 5.15 hardening. <https://lkml.iu.edu/hypermail/linux/kernel/2108.3/06332.html>. Last updated on Aug 30 2021.
22. LLVM. clang 15.0.0 Release Notes. <https://releases.llvm.org/15.0.0/tools/clang/docs/ReleaseNotes.html#major-new-features>. Last updated on Feb 10 2023.
23. Alexey Nurmukhametov M Ramdhan. Exrop Github. <https://github.com/d4em0n/exrop>. Last updated on Feb 21, 2020.
24. Hector Marco-Gisbert and Ismael Ripoll. Return-to-csu: A New Method to Bypass 64-bit Linux ASLR. In *Black Hat Asia 2018*, 2018.
25. Matt Molinyawe, Abdul-Aziz Hariri, and Jasiel Spelman. From Browser to System Compromise. *Pwn2Own 2016*, 2016.
26. Todd Mortimer. Removing ROP Gadgets from OpenBSD. *Proc. of the AsiaBSD-Con*, 2019.
27. OpenBSD. OpenBSD 6.3 Changelog. <https://www.openbsd.org/63.html>. Last updated on May 06 2020.
28. OpenBSD. OpenBSD 6.4 Changelog. <https://www.openbsd.org/64.html>. Last updated on Oct 08 2021.
29. OpenBSD. OpenBSD 6.5 Changelog. <https://www.openbsd.org/65.html>. Last updated on Mar 15 2021.
30. OpenSSH. OpenSSH 9.0 Release. <https://www.openssh.com/txt/release-9.0>. Last updated on Apr 08 2022.
31. Project Zero. Over The Air: Exploiting Broadcom’s Wi-Fi Stack. https://googleprojectzero.blogspot.com/2017/04/over-air-exploiting-broadcoms-wi-fi_11.html. Last updated on April 11, 2017.
32. David Oswald Qais Temeiza. Breaking Bootloaders on the Cheap. *Blackhat Europe*, 2019.
33. ROPgadget. ROPgadget Github. <https://github.com/JonathanSalwan/ROPgadget>. Last updated on Sep 1, 2023.
34. Ropium. Ropium Github. <https://github.com/Boyan-MILANOV/ropium>. Last updated on Jan 10, 2022.
35. Ropium. Ropium speech. https://www.youtube.com/watch?v=rz7Z9fBLVs0&ab_channel=GreHack. Last updated on November 16, 2018.
36. Ropper. Ropper Github. <https://github.com/sashs/Ropper>. Last updated on Jan 12, 2024.
37. Moritz Schloegel. SGC Github. https://github.com/RUB-SysSec/gadget_synthesis. Last updated on Sep 1, 2023.
38. Moritz Schloegel, Tim Blazytko, Julius Basler, Fabian Hemmer, and Thorsten Holz. Towards Automating Code-Reuse Attacks Using Synthesized Gadget Chains. In *Computer Security—ESORICS 2021: 26th European Symposium on Research in Computer Security*, 2021.
39. Felix Schuster, Thomas Tendyck, Christopher Liebchen, Lucas Davi, Ahmad-Reza Sadeghi, and Thorsten Holz. Counterfeit Object-oriented Programming: On the Difficulty of Preventing Code Reuse Attacks in C++ Applications. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015.
40. Edward J Schwartz, Thanassis Avgerinos, and David Brumley. Q: Exploit Hardening Made Easy. In *USENIX Security Symposium*, 2011.
41. Edward J Schwartz, Cory F Cohen, Jeffrey S Gennari, and Stephanie M Schwartz. A Generic Technique for Automatically Finding Defense-aware Code Reuse Attacks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.

42. Hovav Shacham. The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86). In *Proceedings of the 14th ACM conference on Computer and communications security*, 2007.
43. Victor van der Veen, Dennis Andriesse, Manolis Stamatogiannakis, Xi Chen, Herbert Bos, and Cristiano Giuffrida. The dynamics of innocent flesh on the bone: Code reuse ten years later. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1675–1689, 2017.
44. Julien Vanegue, Sean Heelan, and Rolf Rolles. SMT Solvers in Software Security. *WOOT*, 2012.
45. Alexey Vishnyakov. ROP-benchmark Github. <https://github.com/ispras/rop-benchmark>. Last updated on Sep 1, 2023.
46. Alexey V Vishnyakov and Alexey R Nurmukhametov. Survey of Methods for Automated Code-reuse Exploit Generation. *Programming and Computer Software*, 2021.
47. Wei Wu, Yueqi Chen, Xinyu Xing, and Wei Zou. KEPLER: Facilitating Control-flow Hijacking Primitive Evaluation for Linux Kernel Vulnerabilities. In *28th USENIX Security Symposium (USENIX Security)*, 2019.
48. Jianhao Xu, Luca Di Bartolomeo, Flavio Toffalini, Bing Mao, and Mathias Payer. Warpattack: bypassing cfi through compiler-introduced double-fetches. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1271–1288. IEEE, 2023.
49. Z3. Z3 Github. <https://github.com/Z3Prover/z3>. Last updated on Sep 1, 2023.
50. Qing Zhao. Security Improvements in GCC 11 and GCC 12. https://lpc.events/event/11/contributions/1001/attachments/882/1690/LPC_security_gcc_temp.pdf. Last updated on Sep 20 2021.

A Appendix

Table 3: Categorization of instruction semantics based on the type of source operands and destination operands. $\diamond b$ represents any arithmetic operation such as **add** and **xor**.

Semantic Name	Src Operand	Dst Operand	Semantic Definition
MOVE-REG	InReg	OutReg	OutReg $\leftarrow$ InReg
LOAD-CONST	IMM		OutReg $\leftarrow$ IMM
ARITHMETIC-REG	InReg ₁ , InReg ₂		OutReg $\leftarrow$ InReg ₁ $\diamond b$ InReg ₂
LOAD-MEM	M[AddrReg+x]		OutReg $\leftarrow$ M[AddrReg+x]
ARITHMETIC-LOAD-MEM	M[AddrReg+x], OutReg		OutReg $\leftarrow$ M[AddrReg+x]
STORE-REG	InReg	M[AddrReg+x]	M[AddrReg+x] $\leftarrow$ InReg
ARITHMETIC-STORE-REG	InReg, M[AddrReg+x]		M[AddrReg+x] $\diamond b$ $\leftarrow$ InReg
STORE-CONST	IMM		M[AddrReg+x] $\leftarrow$ IMM
ARITHMETIC-STORE-CONST	IMM, M[AddrReg+x]		M[AddrReg+x] $\diamond b$ $\leftarrow$ IMM
MOVE-REG _{pc}	Reg	PC	PC $\leftarrow$ Reg
ARITHMETIC-REG _{pc}	Reg ₁ , Reg ₂		PC $\leftarrow$ Reg ₁ $\diamond b$ Reg ₂
LOAD-MEM _{pc}	M[AddrReg+x] (AddrReg!=sp)		PC $\leftarrow$ M[AddrReg+x] (AddrReg!=sp)
LOAD-STACK _{pc}	M[AddrReg+x] (AddrReg==sp)		PC $\leftarrow$ M[AddrReg+x] (AddrReg==sp)
SYSCALL	Syscall#, SyscallParameters		SYSCALL(SyscallNumber, SyscallParameters)
IF	Condition, Gadget1, Gadget2		if (Condition):PC $\leftarrow$ Gadget1 else : PC $\leftarrow$ Gadget2

A.1 IRB Process

The institution’s IRB application process includes the following steps: (1) initialize application and upload CITI completion reports for all key personnel, (2) complete the protocol document using the template provided by the institution’s IRB office, (3) revise and upload consent documents using provided template, (4) engage in consultations with IRB officers regarding data collection and assessment tools. Revision and re-submission are mandatory in above steps if the institution’s IRB office finds anything ambiguous.

Table 4: Results of *TGRop* and SOTA baselines over ROP benchmark. The meaning of “X/Y/Z” is the same as that in Table 1.

Program #	Debian 10	CentOS 7	OpenBSD 6.2	Total	Debian 10	CentOS 7	OpenBSD 6.2	Total	Debian 10	CentOS 7	OpenBSD 6.2	Total
	139	121	87	347	139	121	87	347	139	121	87	347
	Goal #4: Set Four Registers				Goal #5: Set Five Registers				Goal #6: Set Six Registers			
Exrop	61/0/1	46/4/4	27/0/7	38.62%	21/0/4	13/0/4	10/0/8	12.68%	4/0/2	3/0/3	4/0/8	3.17%
Angrop	41/2/29	29/4/6	8/2/12	22.48%	7/5/34	6/2/5	1/0/3	4.03%	2/1/4	1/1/0	0/0/10	0.86%
ROPium	87/4/1	54/2/0	24/4/1	47.55%	45/2/6	15/1/1	15/2/3	21.61%	12/0/7	6/0/2	10/2/1	8.07%
SGC	22/NA/59	14/NA/39	11/NA/22	13.54%	11/NA/41	11/NA/31	4/NA/12	7.49%	3/NA/34	6/NA/19	3/NA/9	3.46%
TGRop	122/0/1	97/0/0	79/0/1	85.88%	80/0/1	49/0/0	44/0/1	49.86%	50/0/1	26/0/0	26/0/1	29.39%

Table 5: One-Hour ROP Construction: Humans vs *TGRop*. ✓ signifies success, ✕* marks success beyond 1 hour, ✕ indicates failure. “Total/Avg.” shows each participant’s total completed problems and average time per problem in seconds.

	Human A	Human B	Human C	Human D	Human E	Human F	Human G	Human H	Human I	TGRop
smblent	✓/2917	✓/3308	✓/2191	✓/2821	✓/1952	✓/3276	✓/2886	✓/3290	✓/3425	✓/984
wuftp	✓/2166	✕/NA	✕/NA	✓/3320	✓/3398	✓/3198	✓/2857	✕/NA	✕/NA	✓/168
vmd	✓/3054	✕/NA	✕/NA	✓/3249	✓/2720	✓/3502	✕/3814	✕/NA	✕/NA	✓/177
sudo	✓/2796	✕/NA	✕/NA	✕/NA	✕/NA	✕/3808	✕/4161	✓/2730	✕/NA	✓/143
install-info	✕/NA	✕/NA	✕/NA	✕/NA	✕/NA	✕/NA	✕/5737	✕/NA	✕/NA	✓/100
libnetsmp.so.40.2.1	✓/3410	✓/3521	✕/NA	✕/NA	✓/2647	✕/3712	✓/2899	✕/NA	✕/NA	✓/561
mpathpersist	✕/NA	✕/3812	✕/NA	✕/NA	✓/2863	✓/2788	✓/3199	✕/NA	✕/NA	✓/82
mount_nfs	✕/NA	✕/NA	✕/NA	✕/NA	✕/NA	✓/3094	✓/2546	✕/NA	✕/NA	✓/183
ftp	✓/3313	✕/NA	✕/NA	✕/NA	✓/2091	✕/6061	✕/3710	✕/NA	✕/NA	✓/126
libwget.so.2.0.0	✓/2381	✓/3498	✕/NA	✕/NA	✓/2960	✓/2418	✕/3918	✓/1437	✕/NA	✓/584
Total/Avg.	7/2862	3/3535	1/2191	3/3130	7/2662	6/3540	5/3573	3/2486	1/3425	10/310.8

Table 6: Our analysis classified gadgets for the setting about `rdi`, `rsi`, and `rdx`. # Gadget shows the average number of gadgets related to different register sets in the dataset, and each row presents their distribution. **Base** marks existing tool usage. **+Arith** adds gadgets through arithmetic support. **+TGRop-Repair** highlights gadgets unique to *TGRop*’s approach. The results show that conventional heuristic tools only leverage 18% of the gadgets that *TGRop* can effectively utilize.

Test suite	Dst Operand	# Gadgets	From Stack		From Reg		From Imm		From Memory		Total	
			Base	+TGRop-Repair	Base	+Arith +TGRop-Repair	Base	+TGRop-Repair	Base	+Arith +TGRop-Repair	Base+Arith	TGRop-Repair
CentOS 7	<code>rdi</code>	208.26	3.53%	6.31%	1.36%	4.40%	59.53%	8.16%	8.28%	0.78%	0.03%	6.99%
	<code>rsi</code>	129.05	2.92%	5.74%	11.49%	4.67%	53.95%	4.04%	10.56%	0.22%	0.02%	6.36%
	<code>rdx</code>	134.26	0.96%	3.88%	5.31%	8.37%	48.17%	3.54%	9.60%	0.69%	0.20%	19.82%
Debian 10	<code>rdi</code>	404.96	2.18%	6.70%	2.19%	2.60%	66.21%	3.60%	9.45%	0.30%	0.04%	6.73%
	<code>rsi</code>	223.52	3.15%	6.31%	2.37%	2.66%	61.58%	4.17%	10.67%	0.29%	0.16%	8.65%
	<code>rdx</code>	221.53	1.71%	3.65%	4.34%	6.45%	53.31%	5.97%	7.69%	0.76%	0.55%	19.81%
OpenBSD 6.2	<code>rdi</code>	234.12	4.00%	1.80%	1.96%	4.59%	62.74%	6.47%	7.64%	0.23%	0.17%	10.42%
	<code>rsi</code>	299.25	0.79%	1.53%	1.77%	2.96%	64.10%	7.94%	12.12%	0.69%	0.34%	7.78%
	<code>rdx</code>	239.84	0.53%	0.96%	4.62%	8.68%	59.05%	1.67%	8.36%	0.89%	0.10%	15.64%
OpenBSD 6.4	<code>rdi</code>	360.04	4.74%	2.04%	1.62%	0.81%	63.01%	5.34%	7.43%	0.24%	0.03%	14.77%
	<code>rsi</code>	441.55	0.95%	1.46%	2.18%	1.50%	71.63%	2.56%	11.24%	0.52%	0.02%	7.94%
	<code>rdx</code>	268.49	0.44%	0.81%	4.49%	1.96%	65.11%	1.23%	8.30%	0.38%	0.02%	17.27%
OpenBSD 6.5	<code>rdi</code>	29.02	16.92%	3.85%	1.24%	0.73%	52.64%	17.83%	4.48%	0.14%	0.01%	2.15%
	<code>rsi</code>	20.22	3.42%	2.31%	2.30%	2.01%	65.93%	14.94%	3.04%	0.22%	0.01%	5.81%
	<code>rdx</code>	23.76	1.71%	2.59%	28.28%	1.92%	30.96%	1.55%	1.86%	0.11%	0.05%	30.97%
-	avg.	-	3.20%	3.33%	5.07%	3.62%	58.54%	5.93%	8.01%	0.40%	0.12%	11.78%
											18.34%	81.66%

Table 7: Goal #1-3’s results of *TGRop* and SOTA baselines on the dataset with ROP mitigations. The meaning of “X/Y/Z” is the same as that in Table 1.

Program #	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero
	98	240	264	50	98	240	264	50	98	240	264	50
	Goal #1: Get a Shell				Goal #2: Arbitrary Write				Goal #3: Set Three Registers			
ROPgadget	2/0/0	0/0/0	0/0/0	0/0/0	NA	NA	NA	NA	NA	NA	NA	NA
Ropper	3/NA/0	4/NA/0	12/NA/1	4/NA/0	NA	NA	NA	NA	NA	NA	NA	NA
Exrop	1/7/23	0/0/13	0/0/21	3/2/12	1/25/16	0/0/13	0/0/18	3/3/11	38/1/13	140/0/17	38/0/27	8/0/14
Angrop	18/1/1	24/0/10	9/0/3	18/1/0	15/0/5	31/0/14	35/2/5	20/1/0	23/3/3	43/2/13	15/0/1	17/2/0
ROPium	20/2/0	17/0/0	12/1/0	9/0/0	24/1/2	8/1/9	21/0/6	12/0/0	46/0/0	48/0/0	40/1/0	23/2/0
SGC	7/NA/11	1/NA/17	1/NA/34	2/NA/28	18/NA/17	0/NA/29	0/NA/35	17/NA/9	81/NA/10	206/NA/22	0/NA/27	12/NA/4
TGRop	81/0/1	185/0/9	124/0/10	45/0/1	93/0/1	231/0/9	180/0/10	46/0/1	97/0/1	231/0/9	126/0/10	47/0/1

Table 8: Goal #4-6’s results of *TGRop* and SOTA baselines on the dataset with ROP mitigations. The meaning of “X/Y/Z” is the same as that in Table 1.

Program #	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero	OpenBSD 6.4	OpenBSD 6.5	OpenBSD 7.3	GCC fzero
	98	240	264	50	98	240	264	50	98	240	264	50
	Goal #4: Set Four Registers				Goal #5: Set Five Registers				Goal #6: Set Six Registers			
Exrop	25/1/16	128/0/20	18/0/56	8/0/14	10/1/15	1/0/19	0/0/41	4/0/14	7/0/15	0/0/20	0/0/40	1/0/14
Angrop	14/4/3	33/3/12	10/0/1	17/1/0	0/1/3	1/0/13	2/0/1	6/1/0	0/0/3	1/0/13	2/0/35	9/0/0
ROPium	32/1/0	40/0/0	22/3/1	21/2/0	11/1/8	8/0/6	17/0/0	7/1/1	9/0/13	9/0/0	8/0/0	6/0/1
SGC	3/NA/8	2/NA/20	0/NA/19	11/NA/1	0/NA/5	1/NA/19	0/NA/12	5/NA/2	0/NA/5	1/NA/18	0/NA/11	0/NA/6
TGRop	72/0/1	175/0/9	80/0/10	38/0/1	29/0/1	12/0/9	25/0/10	22/0/1	20/0/1	9/0/9	17/0/10	13/0/1